

Annexe A14 — Le chantier LOD, leçon complète

LibraryV3 · Phase G, chantier 1 · de la mesure à la Référence I · octobre 2026

Ce document raconte et explique, dans l'ordre où il a été mené, le chantier qui a donné des niveaux de détail aux rochers de la ceinture d'astéroïdes. Il se lit comme une leçon : chaque étape dit **ce qu'on a fait, pourquoi on l'a fait ainsi**, ce qui **a mal tourné** en route, et ce qu'il faut en retenir. Les erreurs sont gardées volontairement : elles enseignent davantage que les succès.

Prérequis. L'annexe A13 (formules de sélection : ϵ , q , τ) et l'annexe A13 bis (le LOD dans son ensemble : pourquoi, quand, comment stocker). Ce document ne les répète pas ; il les applique, et il dit où l'application les a corrigées.

Plan. § 1 rappelle le principe sur une page. § 2 à § 8 suivent le chantier pas à pas : mesurer, construire les niveaux, la structure de données, le stockage, le chargement, l'intégration, la mesure finale. § 9 recense les processus **retirés ou modifiés** par rapport au plan initial, et pourquoi. § 10 rassemble les contre-exemples, § 11 les dettes et la suite, § 12 sert d'aide-mémoire.

1. Le principe sur une page

L'écran est une grille d'échantillonnage : un point par pixel. Un détail plus fin que l'espacement des pixels coûte du calcul sans rien produire, et fait scintiller l'image. Un LOD est une version **pré-filtrée** du maillage, choisie selon la taille de l'objet à l'écran.

Trois grandeurs suffisent :

- ϵ_k , l'erreur géométrique du niveau k par rapport à L_0 , mesurée **hors ligne**, en unités **locales** du mesh ;
- q , le nombre de pixels par unité locale pour **cette** instance dans **cette** vue : $q = k_{px} \times \text{échelle} / w_{min}$;
- τ , la tolérance en pixels (0,5 px par défaut), réglage de qualité de la **vue**.

Un niveau k est admis si son erreur à l'écran reste sous la tolérance : $\epsilon_k \times q \leq \tau$. On dessine le niveau admis le plus grossier.

À retenir. Le niveau n'est pas une propriété de l'objet, mais de l'objet *dans une vue*. Il se calcule par paire (instance, vue), à chaque frame, et ne se stocke jamais.

2. Mesurer avant de construire (étape 2)

La règle M12 du projet impose d'écrire la règle de décision avant la mesure, et de mesurer avant d'implémenter. La première question n'était donc pas « comment faire le LOD ? » mais « combien de travail le LOD peut-il retirer ? ».

2.1 L'instrument de taille apparente

Un instrument existait déjà depuis le commit 741952a : sous le commutateur LV3_LOD_STATS, RenderView classait chaque paire (instance, vue) non cullée en quatre tranches de rayon apparent (moins de 1 px, 1 à 4, 4 à 16, 16 et plus), et comptait instances et faces par tranche.

X L'instrument mort en silence. En relisant `RenderSystem.cpp`, le bloc de comptage s'est révélé être **dans le code commenté**, celui de l'ancienne boucle d'avant le chantier 2b. Le code actif ne gardait que le calcul des paramètres, jamais utilisés. Un run de comptage aurait affiché des zéros, et l'on aurait pu conclure « aucune instance dans cette tranche ». Le compilateur ne disait rien : une variable inutilisée n'est qu'un avertissement, et seulement quand le commutateur vaut 1.

À retenir. Une instrumentation logée dans un bloc qu'on remplace meurt avec lui. Quand on réécrit un bloc, on fait l'inventaire de tout ce qu'il contenait, pas seulement de ce qu'il calculait.

Le bloc a été réinstallé dans le code actif, au point exact où la sélection viendrait plus tard (après le frustum, avant la bifurcation `Inside / Intersect`), sans les deux compteurs qui ne servaient qu'à dimensionner le 2b.

2.2 Valider l'instrument avant de lire ses chiffres

Deux identités doivent tenir sur chaque frame : la somme des tranches d'instances égale `MeshesTested - MeshesCulled`, et la somme des tranches de faces égale `FacesSubmitted`. Elles ont tenu sur les 240 frames. Un instrument dont on n'a pas vérifié la cohérence ne mesure rien de fiable.

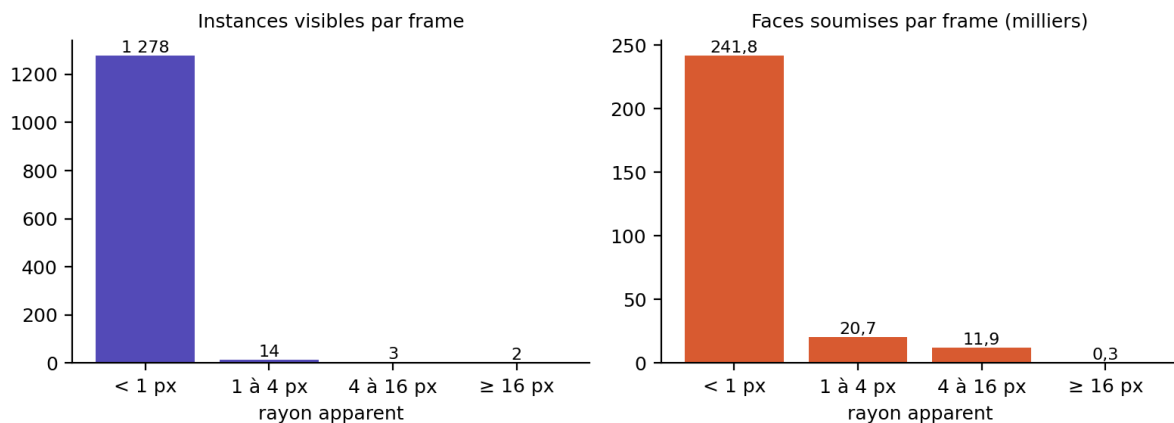


Figure 1 — Run de comptage, ceinture, 4 vues (médiane par frame). 99 % des instances visibles ont moins d'un pixel de rayon et portent 88 % des faces soumises.

2.3 La règle de décision, et une correction en route

Une première estimation assimilait le rayon compté par l'instrument au facteur q de la sélection. C'était trop prudent : l'instrument utilise la sphère circonscrite à l'AABB, dont le rayon vaut environ 1,8 unité locale pour un rocher. Une roche comptée sous 1 px a donc un q inférieur à $1/1,8 \approx 0,56$. Avec $\varepsilon_3 \leq 0,646$ (valeur finale, sécurité comprise), son erreur en L3 reste sous 0,36 px : toutes ces roches descendent en L3.

La règle écrite **avant** l'implémentation : GO si la borne basse garantie dépasse 25 % de `FacesSubmitted` ; prédiction falsifiable : `FacesSubmitted` doit tomber de 275 000 à **107 000 ou moins**, avec au moins **1 053** paires en L3 (borne obtenue en supposant que les autres meshes de la tranche ont au moins 288 faces).

À retenir. On n'a pas eu besoin d'un histogramme plus fin : toutes les roches de la tranche franchissaient le seuil le plus favorable. Avant de raffiner une mesure, se demander si le raffinement peut changer la décision.

3. Construire les niveaux (étape 3)

3.1 Ré-échantillonner la même surface

Les rochers viennent de la fonction `rock()` de `gen_mesher.py`, qui déforme une sphère selon la **normale** de chaque sommet. Une grille plus grossière, avec la même graine, retombe exactement sur la même surface. C'est la meilleure méthode quand elle est possible : exacte, gratuite, sans artefact. Les niveaux : L0 en 12×8 (168 faces), L1 en 8×5 (64), L2 en 4×3 (16), L3 en 4×2 (8, un octaèdre déformé).

Ces niveaux ne sont **pas emboîtés** : les sommets de L1 (tous les 45°) ne sont pas des sommets de L0 (tous les 30°). Conséquence pour le stockage : chaque niveau garde ses propres sommets (dispositions A ou D de l'annexe A13 bis).

3.2 Mesurer ϵ : un maximum échantillonné au hasard est optimiste

ϵ se mesure par écart radial : on lance des rayons depuis le centre, on compare la distance à la surface de L0 et à celle de L_k , on garde le maximum. A13 proposait 400 directions aléatoires. En augmentant l'échantillonnage, ϵ de `rock_a` L1 est passé de 0,235 à 0,344 : **+46 %**.

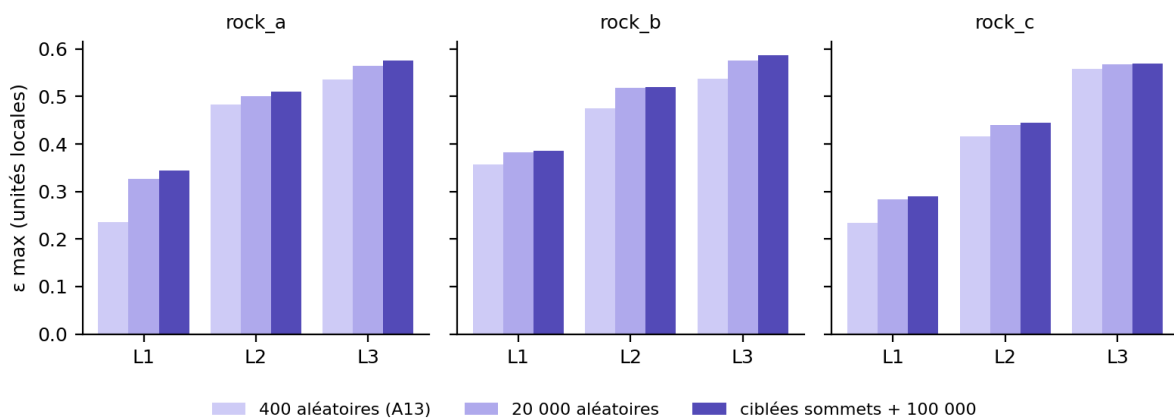


Figure 2 — ϵ max selon l'échantillonnage. Le passage à des directions ciblées sur les sommets fait encore monter L1 ; au-delà, la valeur ne bouge plus.

La raison est géométrique : l'écart entre deux surfaces faites de triangles est maximal près des **sommets**, là où les pentes changent. Un tirage aléatoire passe presque toujours à côté. La mesure finale vise donc chaque sommet des deux maillages (avec un léger décalage pour ne jamais tomber pile sur une arête), plus 100 000 directions aléatoires. La convergence a été vérifiée : de 20 000 à 100 000 directions, rien ne change au-delà du millième.

X Faire confiance à un coefficient de sécurité. A13 multipliait ϵ par 1,1 pour couvrir le sous-échantillonnage. Il aurait couvert 10 %, pas 46 %. La sécurité ne remplace pas une mesure correcte : on échantillonne là où le maximum peut se trouver, on vérifie que la valeur ne bouge plus, et seulement ensuite on applique une marge.

La valeur stockée est la mesure $\times 1,1$, **arrondie vers le haut** au dix-millième : jamais optimiste.

3.3 Le descripteur `.lod.json`

```
{
  "format": "lv3.lodchain", "version": 1, "name": "rock_a", "units": "local",
  "_errorMetric": { "method": "radial", "randomDirections": 100000,
    "vertexTargeted": true, "safety": 1.1 },
  "levels": [
    { "mesh": "rock_a.obj", "vertices": 115, "faces": 168, "epsilon": 0.0, "_measure": {...} },
    { "mesh": "rock_a_L1.obj", "vertices": 52, "faces": 64, "epsilon": 0.3781, "_measure": {...} },
    { "mesh": "rock_a_L2.obj", "vertices": 18, "faces": 16, "epsilon": 0.5616, "_measure": {...} },
    { "mesh": "rock_a_L3.obj", "vertices": 13, "faces": 8, "epsilon": 0.6329, "_measure": {...} } ] }
```

Champ	Rôle	Pourquoi ainsi
format, version	identifie le fichier	un ancien format est refusé au lieu d'être mal lu
units: local	ε avant le Transform	l'échelle s'applique à la sélection, jamais au fichier
mesh	chemin relatif au descripteur	le dossier se déplace sans rien casser
vertices, faces	redondants avec le .obj	détectent un descripteur périmé : erreur explicite au lieu d'un ε faux en silence
epsilon	seule valeur lue pour la sélection	mesure × 1,1, arrondie vers le haut
_errorMetric, _measure	documentation	le préfixe « _ » est la convention de WarnUnread : ni avertissement, ni lecture

Absents volontairement : **τ et les seuils** (ils dépendent de la vue) et **l'AABB** (recalculée au chargement : on ne stocke jamais une donnée qu'on peut recalculer exactement, elle finirait par diverger).

X Compter les sommets du fichier au lieu de ceux du moteur. Le premier chargement a échoué : « 115 sommets, le descripteur en annonce 117 ». La grille écrit $13 \times 9 = 117$ sommets, mais un sommet de chaque pôle n'est référencé par aucune face, et l'OBJLoader ne crée que les sommets cités par les faces. Les deux nombres étaient justes, chacun selon sa définition ; leur comparaison était fausse.

À retenir. Une donnée de contrôle doit être calculée par la **même définition** que la donnée contrôlée. Et la vérification a fait son travail : sans elle, l'écart serait passé inaperçu.

4. La structure de données : LodChain (étape 4a)

La chaîne contient ce que trois moments de RenderView lisent : la boîte englobante (culling), les inverses de ε (sélection), les handles des niveaux (dessin). Tout tient dans une ligne de cache.

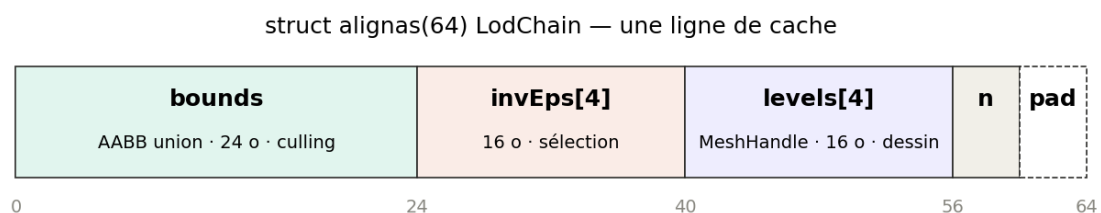


Figure 3 — Disposition mémoire. L'ordre des membres suit l'ordre des lectures. Deux `static_assert` verrouillent la taille (64 octets) et la copie triviale.

```
#pragma warning(push)
#pragma warning(disable : 4324) // remplissage VOULU : 60 -> 64 octets
struct alignas(64) LodChain
{
    static constexpr uint32_t kMaxLevels = 4;
    AABB3d      bounds;           // union des niveaux : culling
    float       invEps[kMaxLevels] = { +inf, -inf, -inf, -inf }; // 1/epsilon : selection
    MeshHandle  levels[kMaxLevels] = {}; // dessin
    uint32_t    levelCount = 0;
};
#pragma warning(pop)
static_assert(std::is_trivially_copyable_v<LodChain>);
static_assert(sizeof(LodChain) == 64);
```

Pourquoi l'union des boîtes. Les sommets de L3 sont posés sur la surface continue, pas sur les triangles de L0 : ils peuvent déborder de la boîte de L0. Culler un objet dessiné en L3 avec la boîte de L0 pourrait rejeter à tort un objet au bord de l'écran.

Pourquoi alignas(64), et le pragma. Sans alignement, des chaînes de 60 octets contiguës chevaucheraient presque toutes deux lignes de cache. MSVC signale le remplissage par C4324 ; on désactive cet avertissement **sur cette structure seulement**, jamais globalement, car il protège ailleurs contre des remplissages non voulus.

4.1 La sélection sans branche

$\varepsilon_k \times q \leq \tau$ s'écrit $q / \tau \leq 1 / \varepsilon_k$, soit $q' \leq \text{invEps}[k]$. Comme ε croît avec k , invEps décroît : les niveaux admis forment toujours un **préfixe** 1, 2, ..., m . Le nombre de niveaux admis est donc exactement l'indice du niveau à dessiner : il suffit de compter.

```
[[nodiscard]] LV3_FORCEINLINE uint32_t SelectLodLevel(const LodChain& chain, float qOverTau) noexcept
{
    uint32_t k = 0;
    for (uint32_t i = 1; i < LodChain::kMaxLevels; ++i)
        k += uint32_t(qOverTau <= chain.invEps[i]); // 3 comparaisons, aucune branche
    return k;
}
```

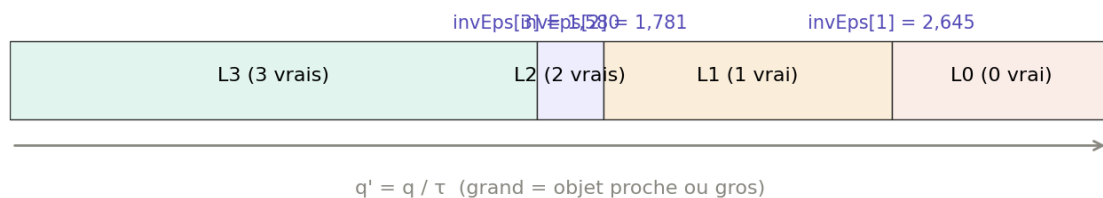


Figure 4 — Pour `rock_a` : chaque région de q' correspond au nombre de comparaisons vraies, qui est le niveau dessiné.

Les cas limites tombent juste sans test : les cases inutilisées valent $-\infty$ (jamais admises, donc une chaîne de longueur 1 rend toujours L0) ; $q' = +\infty$ (objet qui touche le near) et $q' = \text{NaN}$ (donnée corrompue) rendent L0, le niveau le plus fin, jamais un trou.

Attention au nom. `AABB3d::kInf` vaut `numeric_limits<float>::max()`, le plus grand flottant fini, pas l'infini. Ne jamais l'utiliser pour remplir `invEps`.

5. Le stockage dans le ResourceManager (étape 4b)

Les meshes sont rangés dans une `unordered_map` de `unique_ptr` : trois accès mémoire dépendants par lecture. Les chaînes, lues 2 580 fois par frame, sont rangées dans un **`std::vector` indexé directement par l'id du handle** : un seul accès. La case 0 est une sentinelle (l'id invalide).

Trois conséquences, chacune devenue une règle du code :

1. **Le pointeur est temporaire.** Un vecteur qui grandit réalloue : le pointeur rendu par `GetLodChain` n'est valable que jusqu'au prochain enregistrement. Le handle est la référence durable (règle de la leçon 3).
2. **UnloadAll ne vide pas le vecteur.** La taille du vecteur est le prochain id ; la ramener à zéro recyclerait les ids, et un vieux handle désignerait silencieusement une autre chaîne (problème ABA). On vide les cases, on garde la taille.

3. **Une porte d'entrée unique.** RegisterLodChain vérifie tous les invariants dont SelectLodLevel dépend, pour que la boucle de rendu n'en teste aucun.

Invariant vérifié à l'entrée	Ce qu'il protège
$1 \leq \text{levelCount} \leq 4$	les tableaux de taille fixe
$\text{invEps}[0] = +\infty$	L0 toujours admis
chaque niveau a un mesh chargé	aucun pointeur nul au dessin
bounds non vide	un objet invisible au culling, sans message
invEps strictement décroissant et > 0	la propriété de préfixe de la sélection
cases inutilisées à $-\infty$ et handles invalides	une chaîne courte ne dessine jamais un niveau absent

Le test des flottants s'écrit $!(a > 0 \ \&\& \ a < b)$ et non $a \leq 0 \ || \ a \geq b$: avec un NaN, toute comparaison est fautive ; la forme positive niée **rejette** le NaN, la forme directe l'**accepterait**. Cette garantie suppose /fp:precise : sous /fp:fast, le compilateur peut supposer qu'aucun NaN n'existe et supprimer ces tests.

6. Le chargement (étape 4c)

X Deux gardes qui excluent tout. JsonReader::ForEachElement commençait par « si ce n'est pas un objet, sortir », puis « si ce n'est pas un tableau, sortir ». Aucune valeur JSON n'étant les deux, la boucle n'était jamais atteinte. Personne ne l'avait vu : aucun appelant n'aurait encore de tableau. Le chargeur de chaînes en a été le premier.

6.1 Lire, puis décider

LodChainLoader::Load travaille en cinq phases séparées : fichier vers JSON (sans exception) ; en-tête (format, version, unités) ; lecture **brute** des niveaux dans une petite structure, sans rien charger ; construction (charger chaque mesh, comparer ses dimensions, accumuler la boîte, calculer invEps) ; enregistrement par la porte unique. Un descripteur mal formé est refusé **avant le moindre effet de bord** dans le cache.

6.2 Les défauts empoisonnés

Le JsonReader est tolérant par conception : une clé absente donne un avertissement et une valeur par défaut. Pour une chaîne, un défaut plausible serait un mensonge (un ϵ absent remplacé par 0,5 donnerait une chaîne acceptée et fautive). On garde le lecteur tolérant, mais on choisit des défauts qu'**aucune vérification ne laisse passer** : NaN pour epsilon, 0 pour faces et vertices, chaîne vide pour mesh, format vide.

À retenir. Lecteur tolérant + défauts empoisonnés = chargement strict, sans toucher au lecteur. Une clé manquante produit deux messages : celui du lecteur, qui la nomme, et celui du chargeur, qui dit pourquoi la chaîne est refusée.

6.3 Erreurs typées, cache et tests

Chaque échec a sa catégorie (FileNotFound, ParseFailed, BadHeader, BadLevels, LevelMeshFailed, LevelMismatch, Rejected). LoadLodChainChecked reproduit le patron de LoadMeshChecked : clé canonique, cache, délégation. Seuls les succès sont mis en cache : un fichier corrigé sur disque peut être rechargé.

Le test TestLodChain_Load vérifie le chargement, le contenu, la sélection sur des valeurs choisies entre les seuils (y compris $+\infty$ et NaN), le cache (un chemin écrit autrement rend le même handle), le partage de L0 avec le mesh déjà chargé, les chaînes implicites, et **un chemin**

d'échec : un fichier absent doit rendre `FileNotFoundException`. On teste aussi les échecs ; un chargeur dont on n'a vérifié que les succès peut mentir sur la cause d'une erreur réelle.

7. L'intégration (étape 4d)

7.1 Remplacer plutôt qu'ajouter

Deux options pour le `MeshComponent` : ajouter un handle de chaîne à côté du handle de mesh, ou **remplacer** le handle de mesh par un handle de chaîne. L'ajout ne casse rien... et c'est son défaut : tout code qui lisait le mesh continue de compiler et lit L0 en silence (par exemple une boîte englobante calculée sur L0 au lieu de l'union). Le remplacement transforme le **compilateur en outil d'inventaire** : chaque lecteur devient une erreur de compilation, aucun ne peut être oublié.

À retenir. Quand un changement de sens doit se propager partout, changer aussi le **type**. Le membre s'appelle `m_lodChain`, pas `m_lod` : il référence une chaîne, jamais un niveau.

Les meshes sans descripteur (planètes, vaisseau, gizmos) reçoivent une **chaîne implicite** de longueur 1, créée une fois par mesh et partagée (`GetOrCreateSingleLevelChain`), et elle aussi enregistrée par la porte unique. Les `GizmoAssets` portent désormais des chaînes, converties une fois au chargement, puisque `CameraGizmoSystem` échange la chaîne d'un gizmo quand sa projection change. Un test vérifie qu'un gizmo a une chaîne de longueur 1 : un instrument ne se dégrade jamais.

7.2 Trois runs identiques

Le changement de type (4d-2), puis l'arrivée de τ dans la vue (4d-3a), puis la sélection elle-même (4d-3b) ont chacun été suivis d'un run de la ceinture **sans aucun .lod.json dans la scène**. Les compteurs devaient être identiques au run de référence, frame par frame : ils l'ont été trois fois. Toute différence observée ensuite ne pouvait venir que de la descente des roches.

7.3 τ dans la vue

τ suit le chemin de `depthDisplayRange` (règle R12) : défaut dans `engine.json` (section `lod`, et non `debug` : c'est un réglage de qualité), surcharge optionnelle par caméra (`-1` = non défini), résolution dans `BuildViewData`. La vue ne stocke pas seulement τ : elle stocke `lodParams`, une copie des paramètres de taille apparente dont **k_{px} est déjà divisé par τ** . La division par τ est faite une fois par vue, jamais par instance.

7.4 Le nouvel ordre de `RenderView`

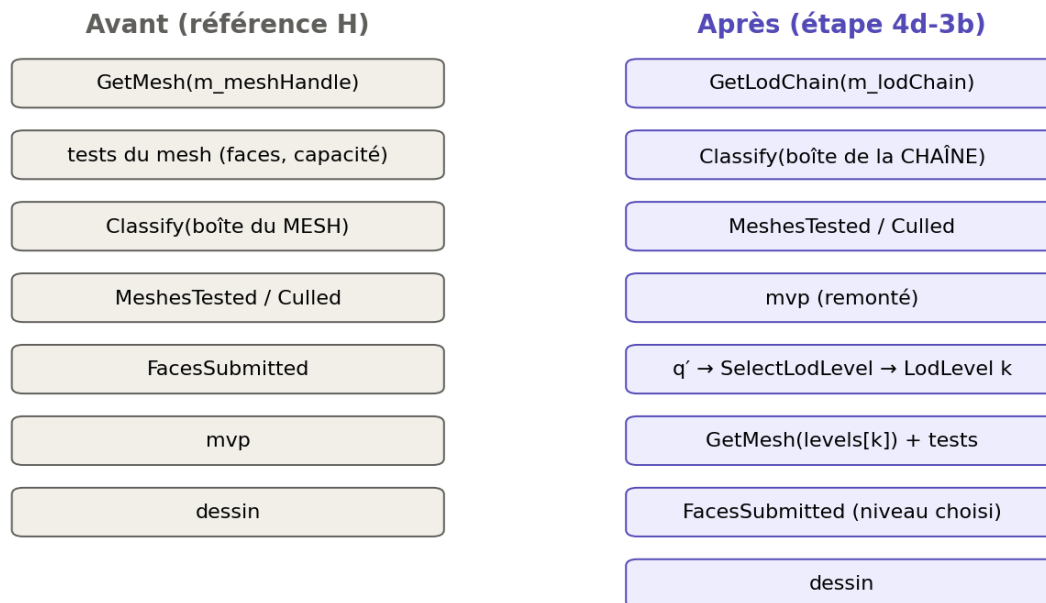


Figure 5 — Chaque étape a besoin de la précédente : la chaîne donne la boîte, le culling évite de sélectionner pour rien, mvp donne w, la sélection donne le mesh, et seulement alors on compte ses faces.

```
const float rW = BoundingRadiusWorld(box, modelMatrix); // rayon local x plus grande echelle d'axe
const float wC = MulRow(mvp, box.Center()).w; // w_clip du centre
// ProjectedRadiusPx(lodParams, ...) = rW * kPx / (tau * wMin) ; divise par le rayon local :
const float qOverTau = ProjectedRadiusPx(view.lodParams, wC, rW) / box.Extent().length();
const uint32_t level = SelectLodLevel(*chain, qOverTau);
LV3_PROF_COUNT(EProfCounter(uint8_t(EProfCounter::LodLevel0) + level), 1);
```

L'échelle non uniforme des roches (par exemple $0,074 \times 0,066 \times 0,060$) est prise par sa plus grande composante : q' est surestimé, la sélection reste du côté fin. L'orthographique fonctionne sans cas particulier (w ne dépend plus de la profondeur).

7.5 La scène le dit explicitement

ParseMesh charge une chaîne si "model" se termine par .lod.json, un mesh simple sinon. Pas de détection d'un fichier voisin : une scène doit se lire sans connaître le contenu du dossier. La scène de référence n'a pas été modifiée ; une copie solar_system_v1compat_belt_lod.json pointe les roches vers leurs descripteurs, et c'est la comparaison des deux qui mesure le LOD.

X Des options de chargement implicites. ParseMesh chargeait les .obj avec flipUVsVertically = false, le chargeur de chaînes avec le défaut (true). Invisible aujourd'hui (le rasterizer ne lit pas les UV), mais les roches en LOD auraient eu leur texture à l'envers. Correction : les options sont définies une fois par la scène et transmises au chargeur, sans valeur par défaut, puisque c'est par un défaut que l'erreur était entrée.

8. La mesure finale

8.1 Correction, puis performance

Un run RelWithAsserts vérifie la **correction** (assertions, invariants), un lot Release mesure la **performance**. Les compteurs ont été identiques entre les deux configurations, et entre deux machines différentes : les assertions n'ont aucun effet de bord, et la simulation est déterministe (pas de temps fixe).

8.2 Un lot refusé, un lot recevable

Un premier lot Release a tourné sur l'i9-10900X, non épinglé, dans l'ordre A B A B. Il ne respectait pas le protocole M13 : dispersion trop forte (p95 ÷ médiane de 1,39 à 1,54), dérive de la machine pendant B2 (témoin Clear +38 %). Il établit que le LOD réussit (−49 %), pas de combien. La Référence H ayant été mesurée sur l'i7-1265U, aucune comparaison de temps entre les deux machines n'est permise (M14).

Le lot recevable : i7-1265U, secteur, start "" /wait /affinity 4, un run jetable, puis A1 B1 B2 A2. Témoins stables (Clear à 3 %, Animation à 6 %, LocalXform1 à 8 %).

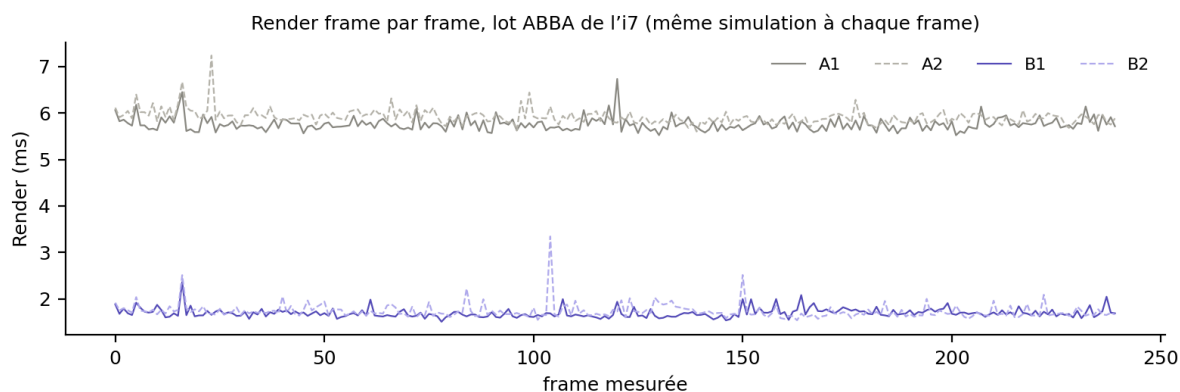


Figure 6 — Render frame par frame. Les deux A se superposent, les deux B aussi : la simulation est identique à chaque frame, seul le LOD change.

	A (sans LOD)	B (avec LOD)	B ÷ A
Render, paire 1	5,738 ms	1,672 ms	0,291
Render, paire 2	5,890 ms	1,688 ms	0,287
Render, ABBA	11,628 ms	3,359 ms	0,289 (−71 %)
Frame, ABBA	15,94 ms	7,75 ms	0,486 (−51 %, ≈ 125 → 258 fps)
FacesSubmitted	274 988	82 988	0,302
Coût par face soumise	21,1 ns	20,2 ns	constant

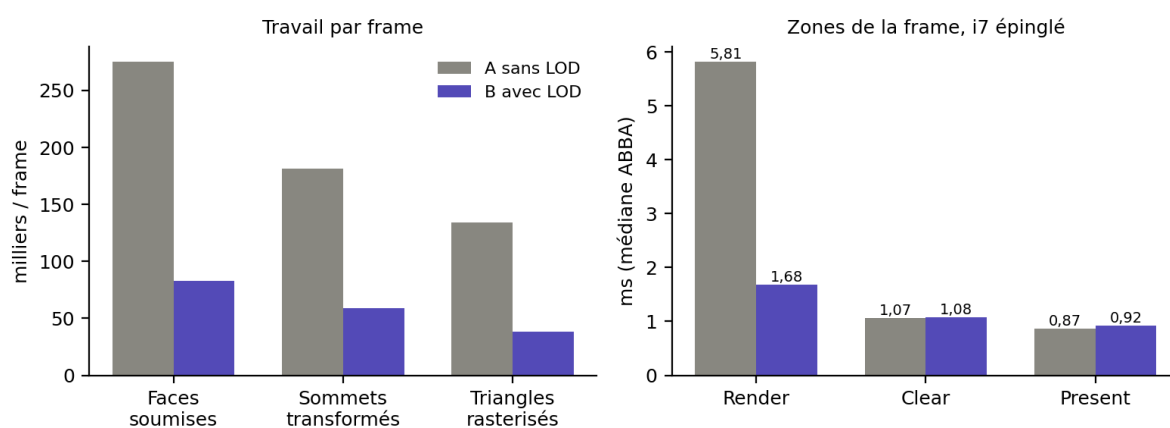


Figure 7 — À gauche, le travail retiré. À droite, la frame : Render passe sous Clear + Present réunis.

8.3 Ce que disent les chiffres

- **La prédiction tient** : 82 988 faces ($\leq 107\,000$), 1 200 paires en L3 ($\geq 1\,053$), aucun popping observé en vol.

- **La comptabilité est exacte** : 192 000 faces retirées = $1\,200 \times (168 - 8)$; 122 400 sommets = $1\,200 \times (115 - 13)$. Aucun effet parasite.
- **La sélection est gratuite** : le coût par face est le même avec et sans LOD. Un surcoût par instance se serait réparti sur trois fois moins de faces en B et l'aurait fait monter.
- **A est 12 à 14 % au-dessus de la Référence H**, et les témoins de calcul aussi (+8 à +14 %) : l'état de la machine a changé entre les séances, pas le code.
- **L1 et L2 ne servent jamais dans cette scène** : toutes les roches visibles sont minuscules. Ils servent quand une caméra s'approche ; le test en vol n'a montré aucune transition visible.

Référence I (i7-1265U épinglé, ceinture, 4 vues, 1536 × 900, Release) : Render = **1,67 à 1,69 ms**, frame = **3,87 ms**, environ **20 ns par face soumise**.

Proposition d'amendement à M13 : la dispersion absolue ($p_{95} - \text{médiane} \approx 0,28 \text{ ms}$) est la même en A et en B ; c'est la grandeur mesurée qui a rétréci. Le critère relatif $p_{95} \div \text{médiane} < 1,15$, calibré pour un Render de 5 à 8 ms, devient trop sévère. Suggestion : recevable si $p_{95} \div \text{médiane} < 1,15$ **ou** $p_{95} - \text{médiane} < 0,3 \text{ ms}$. Décision laissée au projet ; en attendant, la réserve sur B2 (1,166) est déclarée.

9. Processus retirés ou modifiés, et pourquoi

Le plan initial venait des annexes A13 et A13 bis. L'exécution l'a corrigé à plusieurs endroits. Chaque écart ci-dessous est volontaire et justifié ; aucun n'est un oubli.

Prévu	Fait	Pourquoi
$q_{\text{Max}} = \tau / \varepsilon$ précalculé au chargement (A13 § 6.4)	$\text{invEps} = 1 / \varepsilon$ dans la chaîne ; k_{px} / τ dans la vue	τ appartient à la vue ; une chaîne partagée par 4 vues ne peut contenir un seuil qui dépend de la vue. La division par τ reste hors de la boucle chaude.
Séparer données chaudes et froides (A13 bis § 5)	Tout dans une ligne de 64 octets	La règle sert quand les données dépassent une ligne. Séparer une structure qui tient dans une ligne ferait charger deux lignes au lieu d'une.
ε par 400 directions aléatoires, $\times 1,1$ (A13 § 4.2)	Directions ciblées sur les sommets + 100 000 aléatoires, convergence vérifiée, $\times 1,1$ arrondi vers le haut	Le tirage aléatoire sous-estimait ε jusqu'à 46 % ; la sécurité de 10 % ne le couvrait pas.
Retirer un niveau dominé (A13 § 4.5) : L2	L2 conservé	Il ne coûte rien par frame. La scène de mesure n'exerce aucun niveau intermédiaire ; on tranchera avec une scène qui s'approche des roches.
Disposition D (A13 bis § 4.3), ensuite	Disposition A conservée	Aucun besoin mesuré : 3 chaînes, quelques kilo-octets. D reste l'évolution naturelle.
Comptage des tranches dans l'ancienne boucle	Réinstallé dans le code actif, sans VertsInside / TrisEmittedInside ; boîte de la chaîne et faces du niveau choisi	L'instrument était mort ; les deux compteurs du 2b n'avaient plus d'objet ; l'instrument mesure ce que la sélection voit, et $\Sigma \text{FacesPx} = \text{FacesSubmitted}$ reste vrai.
Clés de documentation « errorMetric », « measure »	« _errorMetric », « _measure »	Respecter la convention existante de WarnUnread plutôt que lire de fausses clés.
Sommets comptés dans le fichier	Sommets référencés par les faces	Même définition que le moteur.

Prévu	Fait	Pourquoi
Composant : ajouter un handle de chaîne	Remplacer le handle de mesh	Le compilateur inventorie chaque lecteur ; aucune lecture silencieuse de L0.
Histogramme plus fin des tailles	Abandonné	Toutes les roches de la tranche franchissaient le seuil le plus favorable : la décision ne pouvait pas changer.
Précalculer le rayon local dans le remplissage de la chaîne	Reporté	Optimisation sans mesure ; le coût par face constant a ensuite prouvé que la sélection est négligeable.
Hystérésis et fondu (A13 § 9)	Non implémentés	Aucun popping observé : $\tau = 0,5$ px rend les transitions invisibles.
Niveau « Culled »	Non activé	Faire disparaître des objets change l'image : décision de contenu, pas optimisation.
Options OBJ implicites dans le chargeur	Options de la scène transmises, sans défaut	Le défaut d'OBJLoadOptions diffère de celui de la scène.
Map miroir id → chemin, UnloadLodChain	Non écrits	Aucun appelant. On les ajoutera ensemble le jour où un déchargement individuel servira.
Lot Release A B A B non épinglé	Lot M13 : jetable puis A B B A, affinity 4	Le premier lot n'était pas recevable (dispersion, dérive thermique).

10. Les contre-exemples, rassemblés

X Instrumenter dans un bloc qu'on remplace : l'instrument meurt en silence (§ 2.1).

X Lire un instrument sans vérifier ses identités : un compteur muet ressemble à une tranche vide (§ 2.2).

X Prendre un maximum aléatoire pour un maximum : il est toujours optimiste (§ 3.2).

X Compter la même chose avec deux définitions : 117 contre 115 (§ 3.3).

X Vider le vecteur des chaînes dans UnloadAll : recyclage des ids, ABA silencieux (§ 5).

X Écrire $a \leq 0 \parallel a \geq b$ sur des flottants : un NaN passe (§ 5).

X Deux gardes successives qui excluent chacune la moitié des cas : elles excluent tout (§ 6).

X Des défauts plausibles dans un chargeur strict : une chaîne fausse est acceptée (§ 6.2).

X Ajouter un champ là où il faut changer un type : les anciens lecteurs lisent L0 sans un mot (§ 7.1).

X Une option par défaut là où l'appelant doit décider : textures à l'envers sur les seuls niveaux de LOD (§ 7.5).

X Comparer des temps entre deux machines ou deux séances (M14, § 8.2).

X Ranger un réglage de qualité dans la section debug : quelqu'un le supprimera en production (§ 7.3).

11. Dettes ouvertes et suite

Dette	Où	Quand
Avertissement « clé absente » pour chaque clé optionnelle ; tester Has() d'abord	Serializer	après le chantier LOD
Surcharges caméra (> 0) : une valeur invalide retombe en silence sur EngineConfig	Serializer / BuildViewData	avec la précédente
Cache de LoadMeshChecked indexé par le chemin seul, sans les options	ResourceManager	après le chantier LOD
UnloadMesh devra aussi retirer la chaîne implicite du mesh	ResourceManager	quand UnloadMesh aura un appelant
Clé « graph » de engine.json jamais lue	EngineConfig	à trancher
Hypothèse : coût fixe propre à l'i9 (gain de 49 % contre 71 %)	mesure	lot M13 sur l'i9, un jour

Suite décidée. (1) La chaîne des sphères, en rendant le LOD **optionnel pour tout mesh** : un objet ajouté plus tard doit pouvoir avoir ou non une chaîne, sans code particulier. Les planètes en L0 portent 88 % des faces encore soumises. (2) Clear, devenu presque aussi coûteux que tout le rendu. (3) Les dettes ci-dessus.

12. Aide-mémoire

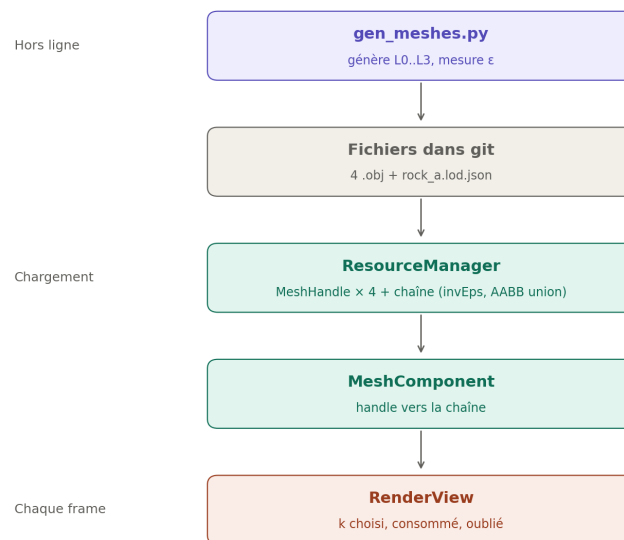


Figure 8 — Trajet des données, du script de génération à la sélection par vue.

Grandeur	Définition	Où
ϵ_k	écart radial max à L0, unités locales, $\times 1,1$ arrondi haut	.lod.json
invEps_k	$1 / \epsilon_k$; $+\infty$ pour L0 ; $-\infty$ pour les cases vides	LodChain
τ	tolérance en pixels (0,5)	engine.json, caméra, ViewData
$\text{lodParams.k}_{\text{px}}$	$P[1][1] \times H/2 / \tau$	ViewData
q'	$\text{ProjectedRadiusPx}(\text{lodParams}, w_c, r_w) / \text{rayon local}$	RenderView

Grandeur	Définition	Où
niveau	nombre de $i \in \{1,2,3\}$ tels que $q' \leq \text{invEps}[i]$	SelectLodLevel
Invariants du profileur	$\Sigma \text{LodLevel} = \text{MeshesTested} - \text{MeshesCulled}$; $\Sigma \text{FacesPx} = \text{FacesSubmitted}$	Profiler.h